# BikeTools Analyzer

## Developer guide

How the code is organised and how to extend it:
new metrics, charts, file formats, services and pages.

Repository `jplanari/BikeToolsAnalyzer`
Applies to the app version shown in the page header (v1.4) and the database metrics version 6.

This guide describes the code as it is in the repository. When it and the code disagree, the code wins; please fix the guide in the same pull request.

# Contents


**1 Introduction** **3**

**2 Getting started** **4**

2.1 Run it from source . . . . . 4

2.2 Run the tests . . . . . 4

2.3 Make a change . . . . . 5

**3 Architecture** **6**

3.1 Layers . . . . . 6

3.2 Repository map . . . . . 6

3.3 How a ride travels through the app . . . . . 7

**4 The data model** **8**

4.1 The standard ride table . . . . . 8

4.2 Cleaning . . . . . 8

4.3 The summary dictionary . . . . . 8

4.4 The database . . . . . 9

4.4.1 Migrations . . . . . 9

4.4.2 Recomputing old rides: `METRICS_VERSION` . . . . . 10

4.5 Reading and writing . . . . . 10

4.6 Headless modules and Streamlit . . . . . 11

**5 Streamlit mechanics** **12**

5.1 Cached reads and `data_stamp` . . . . . 12

5.2 Sections instead of tabs . . . . . 12

5.3 Session state . . . . . 13

5.4 Charts: automatic title and legend placement . . . . . 13

5.5 The look . . . . . 13

**6 Recipes** **14**

6.1 Add a per-ride metric . . . . . 14

6.2 Add an analysis to the Training page . . . . . 15

6.3 Add a section to the ride page . . . . . 16

6.4 Add a chart . . . . . 16

6.5 Read a new file format . . . 17
6.6 Add a profile setting . . . 17
6.7 Add something to the PDF report . . . 17

**7 Connecting a new online service 19**
7.1 Two kinds of secret . . . 19
7.2 Step by step . . . 19

**8 Conventions and pitfalls 22**
8.1 Code conventions . . . 22
8.2 What to test . . . 22
8.3 Common pitfalls . . . 22
8.4 Building the desktop program . . . 23
8.5 Opening a pull request . . . 23

**A Module reference 24**

**B Quick answers 26**

# 1 Introduction

BikeTools Analyzer is a Streamlit application that reads cycling activities (FIT, GPX, TCX, or straight from Strava, Wahoo and Polar), stores them in a local SQLite database and turns them into ride analysis, training-load analytics, physics-based models (aerodynamics, climbing, thermal) and PDF reports. It runs on a rider's own computer, either with `streamlit run` or as a packaged desktop program. All data stays on that computer.

The project is meant to be changed by the people who use it. This guide exists so that a change does not need a long reading session first: it explains the few ideas the whole code base is built on, then gives step-by-step recipes for the changes people most often want.

## What you can add, and where to look

| I want to… | Read | Touches mostly |
|---|---|---|
| add a per-ride number (metric) | §6.1 | `analysis/`, `pipeline.py`, `db/` |
| add an analysis to the Training page | §6.2 | `analysis/`, `viz/training_page.py` |
| add a section to the ride page | §6.3 | `viz/ride_analysis.py` |
| add a chart | §6.4 | `viz/plots.py`, `training_plots.py` |
| read a new file format | §6.5 | `data/`, `pipeline.py` |
| connect a new online service | Chapter 7 | `data/`, `viz/sidebar.py` |
| add a profile setting | §6.6 | `db/users.py`, `viz/user_corner.py` |
| add something to the PDF report | §6.7 | `report_data.py`, `report_pdf.py` |
| change the look | §5.5 | `viz/theme.py` |

## Three rules that make changes safe

1. **Calculations never touch the screen.** Everything under `src/analysis`, `src/physics` and `src/data` is plain Python on pandas objects, with no Streamlit calls. That is what makes it testable and reusable by the PDF report. (A few data modules still import Streamlit for caching; see §4.6.)
2. **The original file is the source of truth.** Every ride's file is stored, so any metric can be recomputed later. A new metric therefore never needs the rider to re-upload anything.
3. **Add a test with the change.** The test suite is quick (under a minute) and runs on every push. Chapter 8 lists what to test.

# 2 Getting started

## 2.1 Run it from source

You need Python 3.10 or newer (the continuous-integration build uses 3.11).

```
git clone https://github.com/jplanari/BikeToolsAnalyzer
cd BikeToolsAnalyzer
python -m venv .venv
.venv\Scripts\activate          # Windows;  macOS/Linux: source .venv/bin/
    activate
pip install -r requirements.txt
streamlit run app.py --server.port 8501
```

Port 8501 matters: the Strava, Wahoo and Polar sign-in redirects point at `http://localhost:8501/`. If another program holds it, Streamlit silently moves to 8502 and sign-in breaks.

The database file is `bikethools.db` in the folder you start from (the spelling is historical). Set the environment variable `BIKETOOLS_DB` to keep it elsewhere, for example outside a OneDrive or Dropbox folder, whose sync clients lock files in ways SQLite dislikes. The desktop build uses its own per-user data folder.

**Use a scratch database while developing**

```
set BIKETOOLS_DB=C:\temp\dev.db          # Windows (cmd)
$env:BIKETOOLS_DB="C:\temp\dev.db"       # Windows (PowerShell)
export BIKETOOLS_DB=/tmp/dev.db          # macOS / Linux
```

Your real rides stay untouched while you experiment.

## 2.2 Run the tests

```
pytest                        # everything
pytest tests/test_upload.py   # one file
pytest -k thresholds -q       # by name
```

Continuous integration (`.github/workflows/python-app.yml`) installs `requirements.txt`, then runs `pytest` on every push. Two kinds of test exist:

- **Headless tests** (most of them) import the calculation and data modules directly. If a heavy library is missing on your machine, `tests/conftest.py` replaces it with a stub, so these tests still run.
- **User-interface tests** (`tests/test_ui.py`, `tests/test_training_page.py`) start the real application with Streamlit's `AppTest`, click through the pages against a seeded database, and fail on any exception. They are skipped locally when Streamlit is not installed, but *required* in CI.

## 2.3 Make a change

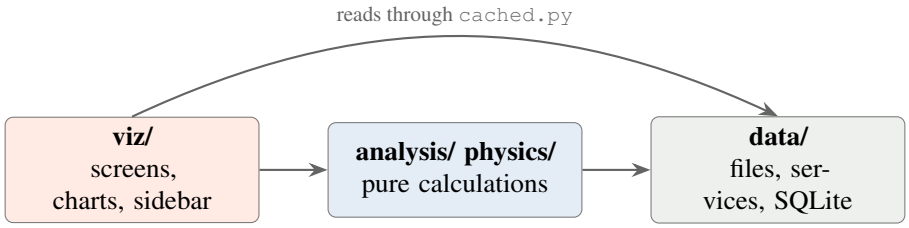
1. Create a branch from `main`: `git checkout -b my-feature`.
2. Make the change and its tests; run `pytest`.
3. Push and open a pull request. CI runs the same tests.
4. To try it as the desktop app, run the *Desktop build* workflow (Actions tab) on your branch.

Keep pull requests focused. A new metric, a new chart and a new service are three pull requests, not one; each is then easy to review and to revert.

# 3 Architecture

## 3.1 Layers

The code is four layers. A module may import from the layers to its right, never from the left.




**src/data**
Everything that is read or written: file parsers (`fit.py`, `gpx.py`, `tcx.py`), service clients (`strava.py`, `wahoo.py`, `polar.py`), the SQLite layer (`db/`) and the *pipeline* (`pipeline.py`) that turns a file into a standard table and its summary numbers.

**src/analysis**
Pure functions: normalized power and TSS (`power.py`), per-ride quality metrics (`ride_metrics.py`), the performance management chart (`training_load.py`), power profile and critical power (`power_profile.py`), climbs, segments, correlations, workout building.

**src/physics**
Models of the rider and the road: aerodynamic drag and CdA (`aerodyn.py`), a climbing model (`climb_model.py`), heat balance (`thermal.py`).

**src/viz**
Everything with Streamlit or Plotly in it: the page for each mode, the sidebar, charts, the look, and the PDF writers.

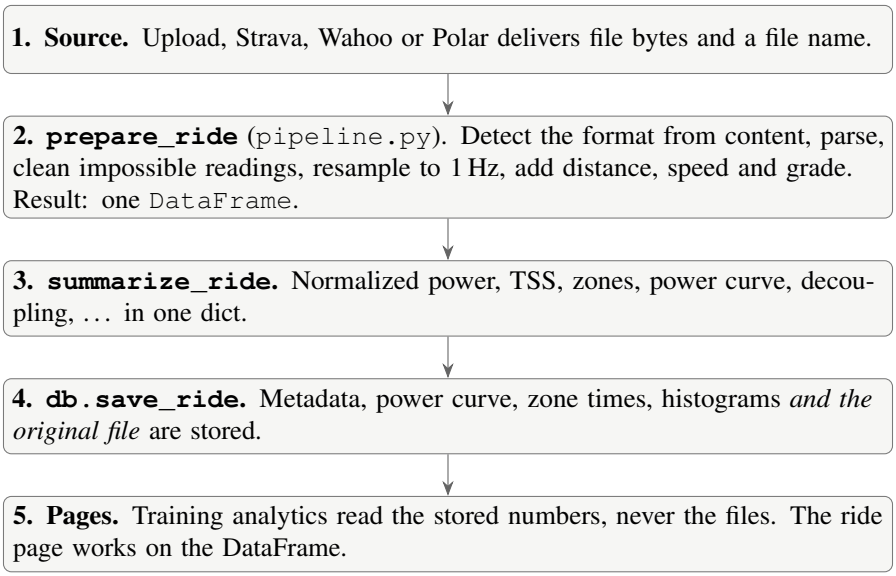
The entry point is `app.py`. It initialises the database, applies the look, finishes any pending sign-in redirect, draws the sidebar, then draws one of three pages according to the top navigation bar.

## 3.2 Repository map

| Path | Role |
|---|---|
| `app.py` | Entry point; chooses the page. |
| `src/viz/sidebar.py` | Users, data sources (upload, Strava, Wahoo, Polar), sign-in callbacks. |
| `src/viz/ride_analysis.py` | The ride page: parse, summarise, save, then draw sections. |
| `src/viz/training_page.py` | The Training page (load, power profile, durability, intensity, deeper). |
| `src/viz/user_corner.py` | Profile, thresholds, history repair, Connections form. |
| `src/viz/connections.py` | In-app entry of service credentials. |

| Path | Role |
|---|---|
| `src / viz / plots . py`, `training_plots.py` | Plotly chart builders. |
| `src/viz/figstyle.py` | Keeps chart titles and legends from overlapping (automatic). |
| `src/viz/sections.py` | Tab-like selector and `remembered()` widgets. |
| `src/viz/theme.py` | Look (CSS), welcome page, help texts. |
| `src / viz / report _ pdf . py`, `ride_report_pdf.py` | PDF writers (matplotlib). |
| `src/data/pipeline.py` | File → standard table → summary dict. |
| `src/data/db/` | Schema, migrations, every read and write. |
| `src/data/cached.py` | Cached reads for the pages. |
| `src/data/credentials.py` | Where service client IDs and secrets come from. |
| `src/data/*_import.py` | Import workflows per source. |
| `src/data/thresholds.py` | FTP and LTHR estimated from data. |
| `desktop/` | Launcher and PyInstaller recipe for the packaged app. |
| `docs/` | OAuth forwarding page, physics report, this guide. |
| `tests/` | One test file per area. |

## 3.3 How a ride travels through the app




Stored numbers make the Training page fast: it reads rows, it does not re-parse hundreds of files. The ride page, which needs second-by-second data, works on the DataFrame from step 2.

# 4 The data model

## 4.1 The standard ride table

`prepare_ride(filename, file_bytes)` returns `(df, total_dist_m, total_ascent_m)`. The DataFrame has one row per second and these columns (a column is absent or all-`NaN` when the device did not record it):

| Column | Meaning | Unit |
|---|---|---|
| `time` | timestamp (naive, UTC) | datetime |
| `lat`, `lon`, `ele` | position and elevation | degrees, m |
| `power`, `hr`, `cadence` | sensor values | W, bpm, rpm |
| `speed` | ground speed | m/s |
| `dist_m` | cumulative distance | m |
| `grade`, `grade_smooth` | road gradient | % |
| `temp_c` | device temperature (FIT only) | °C |
| `bearing` | direction of travel (added by the ride page) | degrees |

**Pitfall**

`speed` is in *metres per second*; a convenience column `speed_kmh` also exists. Mixing the two is the most common source of wrong numbers. Convert at the edge, on display.

Always test for a channel before using it:

```
has_power = 'power' in df.columns and df['power'].notna().any()
```

Indoor rides have no GPS, runs on the same watch have no power, cheap sensors have no cadence; every analysis must degrade gracefully, never raise.

## 4.2 Cleaning

`clean_channels` (in `cleaning.py`) runs before any metric. It removes physically impossible readings: power above 2500 W or negative, heart rate outside 30–230 bpm, cadence above 220 rpm. Short power glitches are interpolated; longer ones become 0 W. The number of samples changed is kept in `df.attrs['cleaned']`. If you add a sensor channel, add its sanity limits there so that one bad sample cannot become a record.

## 4.3 The summary dictionary

`summarize_ride(df, ftp, lthr)` returns the dictionary the ride page shows and the database stores:

| Key | Content |
|---|---|
| `duration_s`, `moving_time_s` | elapsed seconds; seconds with speed $\geq 2$ m/s (or pedalling, if no speed) |
| `avg_speed_kph`, `avg_power`, `avg_hr` | averages over moving time |
| `norm_power`, `if`, `tss`, `tss_source` | load; `tss_source` says whether it came from power or heart rate |
| `kj`, `vi`, `ef`, `decoupling_pct` | work, variability index, efficiency factor, aerobic decoupling |
| `power_curve` | mean-maximal power per duration, `{seconds: watts}` |
| `zones` | seconds per zone: `{'power': {...}, 'hr': {...}}` |
| `fatigued` | power curves of the part of the ride after 1000, 2000 and 3000 kJ of work (durability) |
| `hist` | joint histograms power×HR and power×cadence |
| `hrr60`, `hrr_efforts`, `hr20` | heart-rate recovery; best 20-minute heart rate |
| `is_moving` | boolean mask over the rows of `df` |

`metrics_for_storage(summary)` selects the keys that go to the database. Note the two places where a key is renamed on the way in (`moving_time_s` → column `moving_s`, `hrr60` → `hrr60_bpm`); the mapping lives in `db._metrics_fields`.

## 4.4 The database

SQLite, one file, opened through `db._connect()` only (it turns on foreign keys, so deleting a user cascades to their rides). The schema is created by `init_db()` and extended by `migrate_db()`.

| Table | Holds |
|---|---|
| `users` | name, FTP, LTHR, weight, height, CP, W′, crank length; `ftp_auto`/`lthr_auto` flags |
| `rides` | one row per ride: date, distance, NP, TSS, IF, VI, EF, …, `metrics_version` |
| `ride_files` | the original file, compressed (`codec`, `content`) |
| `power_records` | the ride's power curve, one row per duration |
| `power_records_kj` | the same after a given amount of work |
| `ride_zones` | seconds per zone and kind (`power`, `hr`) |
| `ride_hist` | sparse 2-D histogram cells (`p_hr`, `p_cad`) |
| `planned_sessions` | future sessions (TSS, optional interval blocks as JSON) |
| `strava_auth`, `wahoo_auth`, `polar_auth` | per-rider tokens |
| `app_credentials` | the client ID/secret of each service's API app |
| `oauth_states`, `wahoo_sent` | sign-in tokens; workouts already sent to a head unit |

### 4.4.1 Migrations

There is no migration framework. A new column is added by an `ALTER TABLE` that ignores the error when the column already exists, so it is safe to run on every start:

```
try:
    c.execute("ALTER TABLE rides ADD COLUMN my_metric REAL")
except sqlite3.OperationalError:
```

```
    pass          # already there
```

A new table uses `CREATE TABLE IF NOT EXISTS`. Never rename or drop a column; add a new one and stop reading the old one. Users upgrade by replacing the program and keeping their data folder, so *an old database must always open with new code*.

### 4.4.2 Recomputing old rides: `METRICS_VERSION`

Each ride row stores the `metrics_version` it was computed with. The constant `db.METRICS_VERSION` (currently 6) says what the current code produces. Rides with a lower number are counted as "needing update", the Training page tells the rider so, and *User Corner → Update ride history* re-reads the stored files and recomputes them (`history.recompute_history`), after taking an automatic database backup.

**Rule of thumb**

If your change alters a stored number or adds a stored number, raise `METRICS_VERSION` by one. If it only changes how something is *displayed*, do not.

Rides keep the FTP and LTHR they were scored with (`ftp_at_ride`, `lthr_at_ride`), so changing the profile never rewrites the past.

## 4.5 Reading and writing

- All SQL lives in the package `src/data/db/`. Pages and analysis code call its functions; they never open the database. `fromsrc.dataimportdb` still gives every function: the files are only a split by subject (`core`, `schema`, `users`, `rides`, `stats`, `auth`, `weekly`, `planning`, `bikes`). Put a new function in the file of its subject.
- Functions that write are called directly. Functions that *read for a page* are called through `src/data/cached.py` (next chapter).
- Open a connection with `withconnect()asconn:`. It commits when the block ends, rolls back if it raises, and always closes. Do not call `conn.close()` by hand.
- Duplicate protection: the ride file's hash is unique *per rider* (two riders may save the same club ride), and imports also skip a ride that starts within two minutes of one already saved (the same ride arriving from Strava and from a head unit).

### Schema changes

Never edit a shipped migration. Add a new one at the end of `MIGRATIONS` in `db/schema.py`: a function that receives a connection, registered with the next number. The database keeps its version in `PRAGMAuser_version`; each migration runs once, in its own transaction, after an automatic backup, and a failure rolls back and stops the start with the real error. Adding a column to an existing table is the one thing that does not need a number: call `_add_column(c,table,column,definition)` in `migrate_db`, which checks `PRAGMAtable_info` first.

### Dates and time zones

A ride's start is stored as a UTC instant, always `YYYY-MM-DDTHH:MM:SS+00:00` (`to_utc_iso`); a time without a zone is read as UTC. The *day* a ride belongs to is the rider's: `users.timezone` (an IANA name, set in the User Corner; this computer's zone when empty). The queries that filter by period turn the rider's local day boundaries into UTC (`local_day_bounds_utc`), and what they return in a `date` column, in `get_user_tss_history` and in `start_local` is the rider's own wall-clock time, naive. In tests set `BIKETOOLS_TZ=UTC`; `tests/conftest.py` does.

### Secrets

OAuth tokens, client secrets and the SMTP password are stored as `enc:v1:...` (`secrets_store`); the getters in `db/auth.py` and `db/weekly.py` return plain text, so callers never see the difference. A new secret column needs two things: `encrypt()` when you save and an entry in `_SECRET_COLUMNS`. The key is `BIKETOOLS_SECRET_KEY` or the file `biketools.key` next to the database; do not put that file where the database is synced or backed up.

## 4.6 Headless modules and Streamlit

The goal is that calculation code does not import Streamlit. Two exceptions exist for historical reasons: `gpx.py` caches its parser with `st.cache_data`, and the pipeline imports the parsers. That is why `tests/conftest.py` installs stand-ins for missing libraries and why the import workflows import `pipeline` *inside the function*: it keeps the rest of the module importable without Streamlit. Follow the same habit in new code: import heavy or UI-dependent modules lazily, inside the function that needs them.

# 5 Streamlit mechanics

Streamlit re-runs the whole script from top to bottom every time the user touches a widget. Almost every surprising behaviour in this app comes from that fact. This chapter lists the four mechanisms the code uses to cope.

## 5.1 Cached reads and `data_stamp`

Querying the database and recomputing analytics on every click would be slow with a long history. The module `src/data/cached.py` wraps the read functions of `db/` with `st.cache_data`. The cache key includes `db.data_stamp()`, which changes whenever the database file is written, so anything saved shows up on the next run without manual invalidation.

```
# src/data/cached.py
get_user_rides = _cached('get_user_rides')        # wraps db.get_user_rides
get_power_records = _cached('get_power_records')
```

**Rule:** pages *read* through `cached.py`; code that *writes* calls `db` directly. When you add a new read function to `db/` that a page will call, add one line to `cached.py`.

Analytics that depend only on their inputs (the performance management chart, critical-power history) are cached on those inputs instead, as small functions at the bottom of the same file.

## 5.2 Sections instead of tabs

`st.tabs()` runs the code of *every* tab on every interaction and sends every tab's charts to the browser. Pages here use a horizontal selector and an `if`, so only the visible section is built:

```
from src.viz.sections import section_picker, remembered

sections = ["Load", "Power profile", "My new section"]
section = section_picker(sections, key="training_section")
if section == sections[0]:
    _render_load(...)
elif section == sections[2]:
    _render_my_section(...)
```

Streamlit forgets the value of a widget that is not drawn in a run. A widget inside a section must therefore be created with `remembered()` so its value survives while another section is showing:

```
period = remembered(st.selectbox, "my_period", "90 days", "Period", ["30 days", "
    90 days", "1 year"])
```

Do not pass `index=` or `value=` to the widget; the starting value is the third argument of `remembered`.

The ride page's sections are drawn inside an `st.fragment`, so switching section does not repeat the expensive analysis above it.

## 5.3 Session state

`st.session_state` holds things that must survive a re-run: the chosen user (`current_user`), the file being analysed, a downloaded PDF waiting for its button. It does *not* survive an OAuth redirect (the browser comes back in a new session), which is why sign-in tokens are stored in the database (`db.create_oauth_state`) and not in session state.

## 5.4 Charts: automatic title and legend placement

Every function named `plot_*` in `plots.py` and `training_plots.py` is wrapped, at the end of the module, by `figstyle.tidy_builders(globals())`. The wrapper pins the title to the top-left, puts a horizontal legend under it and grows the top margin to fit both, so titles never overlap the plot or the legend.

- In those two modules, simply name your function `plot_something`; the tidy-up is automatic.
- A chart built anywhere else (for example inside a page) must be wrapped by hand: `st.plotly_chart(tidy(fig))`, importing `tidy` from `src.viz.figstyle`.
- Do not set a manual top margin or title position; they are overwritten.

## 5.5 The look

`theme.py` holds three looks (Compact, Dashboard, Classic) as small CSS packs layered on Streamlit's theme, with colours derived from the text colour so they work in light and dark mode. The accent colour `ACCENT` matches `primaryColor` in `.streamlit/config.toml`; change both together. The welcome page, the FTP/LTHR help texts and the default thresholds are in the same file.

Chart colours are a separate palette in `training_plots.py` (`PALETTES`), with its own light and dark steps; plots read the current mode and call `palette(mode)`.

# 6 Recipes

Each recipe lists the files to touch in order, shows the code, and ends with what to test. They use a running example so the pieces fit together: a made-up metric, *climbing share* (the fraction of moving time spent on a gradient above 3 %).

## 6.1 Add a per-ride metric

A metric that is computed for every ride, stored, and then shown on the ride page and available to the Training page.

### Step 1: the calculation

Put a pure function in the analysis module that fits (here `src/analysis/ride_metrics.py`). It takes pandas objects, returns a number or `None`, and never raises on missing data.

```
def climbing_share(grade, moving=None, threshold_pct=3.0):
    """Fraction of moving time on a gradient above `threshold_pct` (None without
    grade data)."""
    g = pd.Series(grade).to_numpy(dtype=float)
    if len(g) == 0:
        return None
    if moving is not None:
        g = g[pd.Series(moving).to_numpy(dtype=bool)]
    g = g[~np.isnan(g)]
    if len(g) == 0:
        return None
    return float((g > threshold_pct).mean())
```

### Step 2: add it to the summary

In `pipeline.summarize_ride`, add a key to the returned dictionary. Guard on the channel that is needed:

```
'climbing_share': (rm.climbing_share(df['grade_smooth'], is_moving)
                   if 'grade_smooth' in df.columns else None),
```

### Step 3: choose whether to store it

Show-only metrics stop here. To store it (so the Training page can use it without re-reading files):

1. In `pipeline.metrics_for_storage`, add `'climbing_share'` to the `keys` list.
2. In `db.migrate_db`, add `("climbing_share","REAL")` to the list of columns added to `rides`.
3. In `db._METRIC_COLUMNS`, add `'climbing_share'`. The insert (`save_ride`) and update (`update_ride_metrics`) statements are built from this list, so they pick it up.

4. In `db._metrics_fields`, add one entry to the returned dictionary:

```
'climbing_share': _opt_float(metrics.get('climbing_share')),
```

5. Raise `db.METRICS_VERSION` by one so existing rides are offered the recompute.
6. If a page needs it per ride, add the column to the `SELECT` of the matching read function (for example `get_user_rides`); cached readers pick it up automatically.

**Pitfall**

Forgetting step 3.3 stores nothing and raises no error: the column simply stays empty. Forgetting step 3.5 means old rides never get the value. Both are caught by the round-trip test below.

### Step 4: show it

On the ride page, `ride_analysis._render_summary` draws the metric cards in rows of `st.columns`. Add a card to the quality row (`q1…q4`) or open a new row below it:

```
cs = summary.get('climbing_share')
st.columns(4)[0].metric("Climbing share", f"{cs:.0%}" if cs is not None else "N/A
    ",
                        help="Share of moving time on a gradient above 3 %.")
```

Show `"N/A"` rather than `0` when the value cannot be computed; zero is a real result.

### Step 5: test

```
# tests/test_ride_metrics.py
def test_climbing_share_counts_only_moving_seconds():
    grade = pd.Series([0, 5, 5, 0])
    moving = pd.Series([True, True, False, True])
    assert rm.climbing_share(grade, moving) == pytest.approx(1/3)

def test_climbing_share_without_data_is_none():
    assert rm.climbing_share(pd.Series([], dtype=float)) is None
```

And, if stored, a round trip through the database using the `temp_db` fixture of `tests/test_db.py` (save a ride with the new key in its metadata, read the column back).

## 6.2 Add an analysis to the Training page

1. **Calculate.** A pure function in `src/analysis/` that takes the tables the page already reads (rides, power records, zone rows) and returns a DataFrame or dictionary. Example shape: `training_load.compute_pmc`.
2. **Read.** If you need new rows from the database, add a function to the matching file of `src/data/db/` and one line to `cached.py`.
3. **Chart.** Add `plot_my_chart(data, pal)` to `training_plots.py` (§6.4).
4. **Page.** In `training_page.py`, write `_render_my_section(user_name, today, mode)` and add it to the `sections` list and the `if/elif` chain in `render_training_page`:

```
def _render_my_section(user_name, today, mode):
    recs = get_power_records(user_name)                    # cached read
    if recs is None or recs.empty:
        st.info("No power data yet. Import a ride with a power meter.")
        return
    data = my_analysis(recs, today)                        # pure function
    _chart(tp.plot_my_chart(data, tp.palette(mode)))
```

Always handle the empty case with a plain message that says what the rider can do about it; an empty chart or a traceback is the failure users see first. The PDF report uses the same analysis functions (§6.7), so keep the calculation out of the page.

## 6.3 Add a section to the ride page

In `ride_analysis._render_sections`, add the label to `sections` (conditionally, if it needs a channel) and an `elif` branch that calls your renderer with the DataFrame and summary:

```
if has_power and 'lat' in df.columns:
    sections.append("My section")
...
elif section == "My section":
    _render_my_section(df, summary, settings)
```

The renderer receives the 1 Hz DataFrame, so it can do second-by-second work. Anything slow (a physics fit, a weather request) should be computed once in `process_and_display_analysis`, next to `run_heavy_analysis`, and passed down, not recomputed inside a section that re-runs on each click.

## 6.4 Add a chart

Chart builders return a Plotly figure and do not call Streamlit. Follow the conventions already in `training_plots.py`:

- **One y-axis per chart.** Two measures of different scale are two charts.
- **Colours from `palette(mode)`**, in series order; never hard-code hex values in a builder. Text uses neutral ink colours, never the series colour.
- **A legend whenever there is more than one series**; none for a single series (the title names it).
- Name the function `plot_*` so title and legend placement is automatic (§5.4).
- Return an empty-but-valid figure, or let the page show a message, when there is no data.

```
def plot_climbing_share(df, pal):
    """Climbing share per month (bars)."""
    fig = go.Figure(go.Bar(x=df['month'], y=df['share'] * 100, name="Climbing
    share",
                           marker_color=pal['series'][0]))
    fig.update_layout(title="Climbing share per month", yaxis_title="% of moving
    time")
    fig.update_xaxes(showgrid=False)
    return fig
```

Test that the figure builds from a small DataFrame (`tests/test_plots.py` has examples) and, if there is a decision in it (a threshold, a colour rule), test the decision as a separate pure function.

## 6.5 Read a new file format

1. **Parser.** Add `src/data/myformat.py` with `parse_myformat(file_bytes)` returning a DataFrame with at least a `time` column and, where available, `lat`, `lon`, `ele`, `power`, `hr`, `cadence`, and `speed` in m/s. Look at `tcx.py`, the shortest example. Raise `ValueError` if the file holds no timestamped samples.
2. **Recognise it.** In `upload.sniff_kind`, add a test on the first bytes of the file, and add the kind to `ACTIVITY_KINDS`. The type is decided from the *content*; file names are not trusted (a file exported as `ride` or `activity.xml` must still work).
3. **Route it.** In `pipeline.ride_kind` return the new kind, and in `pipeline.prepare_ride` add the `elif` that calls your parser. If your format carries a reliable speed channel, add its kind to the tuple in the "derive speed" rule so speed is not recomputed from GPS.
4. **Accept it.** Add the extension to `upload.UPLOAD_TYPES` (the uploader's allowed types).
5. **Sport filter.** The bulk importer skips non-cycling files using the sport recorded inside the file (`upload_import._sport`); add a branch there if your format records it.

Test with a tiny synthetic file built in the test itself (see `tests/test_upload.py`), including a damaged one: the parsers are expected to return what they can rather than fail the whole upload.

## 6.6 Add a profile setting

A per-rider value, such as crank length (already present) or a second threshold.

1. **Column.** `ALTER TABLE users ADD COLUMN ...` in `migrate_db` with a default (§4.6 showed the pattern).
2. **Read and write.** `get_user_data` returns the row as a dict, so the new field appears in it; add a small writer such as `update_user_crank`.
3. **Edit it.** Add a control in `user_corner.py`. A control that writes should call the writer and `st.rerun()`.
4. **Use it.** Pass it through the `settings` dictionary built at the end of `render_sidebar` if the ride page needs it.

Because every cached read is keyed on the database file's stamp, the page shows the new value on the next run without clearing anything.

## 6.7 Add something to the PDF report

The report is built in two steps on purpose, so the screen and the paper cannot disagree.

1. `report_data.collect_report_data(user, period_days, today)` gathers every number (no plotting, no Streamlit). Add your analysis there and put the result in the returned dictionary; return an empty table, never `None`, when there is no data.

2. `report_pdf.build_report_pdf(data)` draws pages with matplotlib and returns the PDF as bytes. Add a drawing function and call it from the page sequence, checking `.empty` first.

The single-ride report works the same way (`ride_report_data.py`, `ride_report_pdf.py`). Reports are built only when the rider clicks, because matplotlib is slow.

# 7 Connecting a new online service

Strava, Wahoo and Polar are connected the same way, so a fourth service is a matter of copying a pattern. Polar is the simplest to copy (its tokens never expire); `polar.py`, `polar_import.py` and `tests/test_polar.py` are the reference implementation. Suppose the service is called `acme`.

## 7.1 Two kinds of secret

**App credentials** The *client ID* and *client secret* of the API application someone registered with the service. One set per installation. Entered in the app (stored in table `app_credentials`) or written to `.streamlit/secrets.toml`; what is entered in the app wins (`credentials.resolve`).

**Rider tokens** The access (and refresh) token each rider obtains by signing in. One set per rider and service, in table `acme_auth`.

**Never commit secrets**

A client secret must not appear in the repository, in a test, or in the packaged desktop program. Tests use fake values; the real ones live in the user's data folder. The same applies to tokens in log messages and error texts.

## 7.2 Step by step

### 1. The client: `src/data/acme.py`

No Streamlit, no database. It holds the configuration, the errors, the OAuth calls and the API calls:

```
@dataclass
class AcmeConfig:
    client_id: str
    client_secret: str
    redirect_uri: str = "http://localhost:8501/"

class AcmeError(Exception): ...
class AcmeAuthError(AcmeError): ...          # token revoked: rider must reconnect
class AcmeRateLimitError(AcmeError): ...     # HTTP 429

def authorize_url(cfg, state): ...           # where the Connect button sends the
    rider
def exchange_code(cfg, code): ...            # one-time code -> {'access_token',
    ...}
def list_activities(access_token): ...       # summaries
def download_file(access_token, activity_id): ...   # bytes of a FIT/GPX/TCX file
def is_cycling(activity): ...
def ride_filename(activity_id): return f"acme_{activity_id}.fit"
```

Translate HTTP failures into your error classes in one private `_request` helper, as `polar.py` does, and always pass a `timeout`. Read the service's terms: some forbid storing data beyond a period, sharing it, or exceed what a free tier allows; put the relevant limits in the module docstring.

### 2. Store the rider's token

In `db/auth.py`, create `acme_auth` in `migrate_db` (copy `polar_auth`; encrypt the token with `secrets_store.encrypt`) and add `save_acme_auth`, `get_acme_auth`, `delete_acme_auth`. Add `get_imported_acme_ids(user)` and `acme_id_from_filename` on the model of the Polar pair: the *file name carries the service's activity ID*, which is how a repeated sync knows what it already has.

### 3. The import workflow: `src/data/acme_import.py`

Functions the sidebar will call: `get_auth`, `connect`, `list_recent_rides`, `fetch_ride_file`, `sync_new_rides`. The last returns the shared `ImportResult` (`strava_import.py`), and must be:

- **Idempotent**: running it twice imports nothing the second time;
- **Tolerant**: a file that cannot be read goes into `res.failed` and the loop continues;
- **Duplicate-aware**: skip a ride whose start is within two minutes of one already saved (`_is_near`); the same ride often arrives from several sources;
- **Polite**: stop with `res.stopped_reason` on a rate limit, never loop.

Reuse `pipeline.prepare_ride`, `summarize_ride`, `metrics_for_storage`, `db.build_ride_metadata` and `db.save_ride` exactly as `polar_import.sync_new_rides` does, so an imported ride is indistinguishable from an uploaded one.

### 4. Register the credentials

Add an entry to `credentials.PROVIDERS`:

```
"acme": {
    "label": "Acme",
    "portal": "https://developers.acme.example/apps",
    "redirect": "http://localhost:8501/",
    "steps": "Create an application and add the redirect address below to it.",
},
```

The in-app form, the *User Corner* → *Connections* list and the resolution order then work for the new service without more code. In `sidebar.py` add the config function next to the others:

```
def _acme_config():
    c = get_credentials("acme")
    return ac.AcmeConfig(c["client_id"], c["client_secret"], c["redirect_uri"])
    if c else None
```

### 5. The sidebar and the redirect

1. Write `_render_acme_sidebar(user_name, ftp, lthr)` on the model of `_render_polar_sidebar`: show `setup_box("acme")` when the config is missing; a "Connect" link button when the rider has no token; a list of recent rides, "Analyze" and "Sync new rides" buttons when connected; "Disconnect". It returns a file-like object with a `.name` (an `io.BytesIO` with the file name attached) for the ride page, or `None`.

2. Use a state prefix. The OAuth `state` sent to the service is `"acme." + token`, and `handle_strava_callback` routes on the prefix to a `_finish_acme_callback`. The prefix is the only way the shared redirect page knows which service is calling back.
3. Add `"Acme"` to the `st.sidebar.radio("Source", ...)` list in `render_sidebar` and an `elif` that calls your renderer.

**Services that demand https**

Some services refuse an `http://localhost` redirect. The repository already contains a static forwarding page, `docs/oauth/index.html`, published with GitHub Pages, that receives the https redirect and passes the code on to `localhost:8501`. Register that address with the service and put it in the `redirect` field above. It sees only the one-time code, which is useless without the client secret.

### 6. Tests and documentation

Test the client with a fake HTTP response (`FakeResponse` in `tests/test_strava.py`) and monkey-patched `requests`; test the import workflow against the `temp_db` fixture with a stubbed client, covering: a new ride is saved, a second sync imports nothing, a near-duplicate is skipped, an unreadable file is reported without stopping the loop, a rate limit stops cleanly. Then add a section to the README (one-time setup, limits) and an example block to `.streamlit/secrets.toml.example`.

- ☐ client module, no Streamlit
- ☐ auth table and three functions; imported-ID helper
- ☐ import workflow returning `ImportResult`
- ☐ entry in `credentials.PROVIDERS` and a config function
- ☐ sidebar renderer, state prefix, callback branch, `Source` radio entry
- ☐ tests for client and import; README and secrets example

# 8 Conventions and pitfalls

## 8.1 Code conventions

- **Pure first.** Put the calculation in a function that takes data and returns data; the page only calls it.
- **Docstrings say what and why**, in plain words, including the units and what is returned when there is no data. Many modules open with a short essay on the reasoning; keep that habit when the method is not obvious.
- **Named constants** for every threshold (`MOVING_THRESHOLD_MS`, `MAX_POWER_W`); no bare numbers inside formulas.
- **Missing data is normal.** Return `None` or an empty frame; the page decides what message to show.
- **Time is UTC.** Convert to local time only for display.
- **Units at the edge.** Compute in SI (m, m/s, W); convert to km/h only when printing.
- **Lazy imports** of anything heavy or UI-bound, inside the function that needs it.
- **No new dependency lightly.** Packages go in `requirements.txt` with a lower bound that the code needs and an upper bound below the next major release; the desktop build bundles every one of them.

## 8.2 What to test

| You changed | Test |
|---|---|
| a calculation | known input → hand-computed output; the empty and the one-sample case |
| a parser | a tiny synthetic file built in the test; a truncated one; a file without power |
| the database | round trip through the `temp_db` fixture; opening an *old-schema* database |
| a service client | fake responses for success, 401, 404, 429, network error |
| an importer | idempotence, duplicate start, unreadable file, rate limit |
| a page | the `AppTest` smoke tests must still pass; add a click-through for a new section |

## 8.3 Common pitfalls

**Widgets that lose their value**

A widget in a section that is not drawn forgets its value. Use `remembered()`.

**Stale caches in tests**

Cached reads survive between tests unless you call `st.cache_data.clear()` (the UI tests do, in their fixture).

**Seconds versus samples**

Everything downstream of `prepare_ride` assumes one row per second. A function that works on 5-second data must resample first; do not assume the source rate.

**Editing a stored column's meaning**

If you change what an existing stored number means, raise `METRICS_VERSION`, or old and new rides will silently disagree in the Training charts.

**Mutating the DataFrame**

Cached or shared DataFrames must not be changed in place. Copy first (`df = df.copy()`) in any function that adds columns.

## 8.4 Building the desktop program

The packaged app is built by PyInstaller from `desktop/biketools.spec`; `desktop/launcher.py` starts the Streamlit server on port 8501 and opens a window (falling back to the browser). The *Desktop build* workflow runs it on Windows, macOS and Linux. Things to remember when you change the code:

- A new *data file* the program reads at run time (a template, an image) must be listed in the spec; PyInstaller only bundles imports it can see.
- A module imported only by name (`importlib`) or lazily may need a "hidden import" entry.
- The user's data lives outside the program folder (`%APPDATA%\BikeToolsAnalyzer` on Windows), so updating means replacing the program folder and keeping the data.
- Builds are unsigned: Windows SmartScreen and macOS Gatekeeper warn on first start.

## 8.5 Opening a pull request

- ☐ tests added or updated, `pytest` passes
- ☐ no secrets, no personal ride files, no database in the commit
- ☐ `METRICS_VERSION` raised if stored numbers changed
- ☐ new read function added to `cached.py` if a page uses it
- ☐ README or this guide updated if behaviour or structure changed
- ☐ pages show a clear message when data is missing

# A Module reference

| Module | Main entry points |
|---|---|
| **src/data** | |
| pipeline.py | prepare_ride, summarize_ride, metrics_for_storage, moving_mask, ride_kind |
| db/ | init_db, save_ride, update_ride_metrics, get_user_rides, get_power_records, backup_db, data_stamp, METRICS_VERSION |
| cached.py | cached versions of the page reads; compute_pmc, cp_history |
| history.py | recompute_history, ftp_used_for_ride |
| thresholds.py | estimate_ftp, estimate_lthr, refresh_auto_thresholds |
| cleaning.py | clean_channels |
| fit.py, gpx.py, tcx.py | parsers; compute_distance_and_ascent, resample_to_seconds, compute_grade |
| upload.py, upload_import.py | unpack zip/gz uploads; import_activities |
| strava.py, wahoo.py, polar.py | service clients |
| *_import.py | per-service import workflows, ImportResult |
| wahoo_send.py | send a planned workout to a Wahoo head unit |
| credentials.py | PROVIDERS, resolve |
| weather.py | wind and temperature for a ride (Open-Meteo) |
| report_data.py, ride_report_data.py | numbers for the PDF reports |
| **src/analysis** | |
| power.py | NP, IF, TSS, power_curve, coggan_zones, estimate_critical_power |
| ride_metrics.py | variability_index, efficiency_factor, aerobic_decoupling, training_stress, zone_seconds |
| training_load.py | compute_pmc (CTL 42 d, ATL 7 d, TSB) |
| power_profile.py | records, critical-power history, suggest_threshold |
| durability.py | fatigued_curves (after 1000/2000/3000 kJ) |
| intensity.py, correlations.py | intensity distribution; power-HR and power-cadence relations |
| climbs.py, segments.py | climb detection; segment analysis |
| workout.py, wahoo_plan.py | structured workouts and their export format |
| labels.py, dates.py | plain-language status texts; date parsing |
| **src/physics** | |
| aerodyn.py | power balance, CdA fit |
| climb_model.py | predicted climb time from power and mass |
| thermal.py | heat-balance model of the rider |
| **src/viz** | |
| sidebar.py | render_sidebar, handle_strava_callback |
| ride_analysis.py | process_and_display_analysis |
| training_page.py | render_training_page |
| user_corner.py | render_user_corner, render_history |
| connections.py | get_credentials, credentials_form, setup_box |

| Module | Main entry points |
|---|---|
| `plots.py`, `training_plots.py` | `plot_*` chart builders, `palette` |
| `figstyle.py` | `tidy`, `tidy_builders` |
| `sections.py` | `section_picker`, `remembered` |
| `theme.py` | look, welcome page, help texts |
| `report_pdf.py`, `ride_report_pdf.py` | `build_report_pdf` and the single-ride equivalent |

# B Quick answers

**The Training page says some rides are outdated.**
Their `metrics_version` is below the current one. Run *User Corner* → *Update ride history*.

**A new metric is empty for old rides.**
Raise `METRICS_VERSION` and recompute.

**My change does not show up after saving.**
A cached read is stale only if the database file was not written; check that the write goes through `db/`. Restart `streamlit run` after editing a module the server already imported.

**A chart title overlaps the legend.**
The builder is not named `plot_*`, or it is outside the two chart modules; wrap it with `tidy()`.

**Sign-in goes to the wrong port.**
The app started on 8502; free port 8501.

**Where is the rider's data?**
Source run: the database file in the folder you started from (or `BIKETOOLS_DB`). Desktop app: the per-user data folder.